

Self-Reflective Report

Student Name: Zineb Afqir

Student ID: 202505010522

Course: 202605F1923 / BIT1123 - Object Oriented Programming

Assignment: Assignment 1 - Individual

1. Introduction

This report reflects on my learning journey throughout the Object Oriented Programming (OOP) tutorials. The course introduced me to the core principles of Java programming and object-oriented design. Over ten weeks, I progressed from writing my first simple Java program to building a fully interactive graphical user interface using Swing. This report summarises the knowledge I gained from each tutorial, the challenges I encountered and how I overcame them, the improvements I made in my Java programming skills, my understanding of OOP concepts, my future learning plans, and a final conclusion.

2. Knowledge Gained from Each Tutorial

Week 1 - Java Fundamentals

I learned the basic structure of a Java program. I created my first **HelloWorld** program, which taught me about the **main** method, packages, and console output using **System.out.println**. This was my foundation for understanding how Java programs are written and executed.

Week 2 - Classes and Objects

I learned the difference between a class and an object. I created a **Student** class that models a student's name, age, and GPA. I used constructors to initialise objects and methods to display information and represent behaviours such as studying and taking an exam.

Weeks 3-4 - Inheritance and Polymorphism

I learned how to create a **Person** superclass and then build **Student** and **Lecturer** subclasses that extend it. By overriding the **introduce()** method in each subclass, I demonstrated inheritance, method overriding, and runtime polymorphism. I also learned how superclass references can hold subclass objects.

Week 5 - Encapsulation

I learned how to protect data by making fields private and providing controlled access through getter and setter methods. I applied this to a **student** class with private student ID, name, CGPA, and programme fields, and I documented why encapsulation improves data protection and maintainability.

Week 6 - Inheritance and Constructor Chaining

I built an **Employee** superclass and a **Lecturer** subclass. I learned how the **super()** keyword calls the parent constructor and how constructor chaining works. This tutorial reinforced code reuse and the use of protected access modifiers.

Week 7 - Abstraction and Method Overriding

I created an abstract **Appliances** class that defines shared power behaviour and an abstract **operate()** method. I then implemented **Microwave**, **Refrigerator**, **WashingMachine**, and **AirConditioner** subclasses, each providing its own behaviour. This demonstrated abstraction, inheritance, and polymorphic method calls.

Weeks 8-9 - Collections, File Handling, and Exceptions

I built a console task manager that stores user input in an **ArrayList**. I learned how to write tasks to a file and read them back using buffered streams. I also handled **IOException** errors using **try-catch** blocks and try-with-resources, which taught me robust error handling.

Week 10 - Java Swing GUI

I built a Programming Quiz Battle application using Swing components such as **JFrame**, **JLabel**, and **JButton**. I implemented the **ActionListener** interface to respond to button clicks and separated quiz data and answer checking into a separate **Questions** class. This was my introduction to event-driven programming.

3. Challenges Encountered

- Understanding the difference between classes and objects, and knowing when to create new classes.
- Understanding how inheritance works and how to use the **super()** keyword correctly.
- Distinguishing between method overloading and method overriding.
- Managing file input and output correctly, especially handling exceptions and closing resources.
- Designing the layout of the Swing GUI and wiring up buttons to respond to user actions.
- Confusing constructor syntax and making sure parent constructors were called in the right order.

4. How the Challenges Were Overcome

- I reviewed the lecture notes and tutorial examples repeatedly until the concepts were clear.
- I wrote small test programs to experiment with each new idea, such as calling constructors and overridden methods.
- I used the IDE's debugging tools to step through my code and see exactly what was happening.
- I consulted the official Java documentation and online resources for the correct syntax for file handling and Swing.
- I asked my lecturer and classmates for help and discussed problems with them.
- I practised rewriting the programs from memory, which helped the concepts become automatic.

5. Improvements in Java Programming Skills

- I can now write complete Java programs from start to finish without following step-by-step instructions.
- I understand and use proper naming conventions and code structure.

- I am comfortable reading and understanding compiler errors and stack traces, and I can fix errors quickly.
- I can organise code into multiple classes and files instead of putting everything in one file.
- I can use the Java Collections Framework and perform file operations with confidence.
- I can build simple desktop applications with a graphical interface.
- My debugging and problem-solving skills have improved significantly.

6. Understanding of Object-Oriented Programming Concepts

Before this course, I had no real understanding of object-oriented programming. Now I understand that OOP is about modelling real-world things as objects that have attributes and behaviours. I have applied all four pillars of OOP:

- **Encapsulation:** protecting data with private fields and exposing it through getters and setters.
- **Inheritance:** creating subclasses that reuse and extend the behaviour of a superclass.
- **Polymorphism:** using superclass references to call overridden methods of different subclass objects.
- **Abstraction:** defining abstract contracts that force subclasses to implement required behaviour.

The most valuable lesson is that OOP is not only about syntax; it is about assigning responsibilities to classes, controlling access to data, and designing software that is easy to extend and maintain.

7. Future Learning Plans

- I want to learn more advanced Java topics, such as interfaces, generics, and lambda expressions.
- I plan to study the Model-View-Controller (MVC) pattern for building larger applications.
- I want to explore databases and learn how to connect Java programs to them using JDBC.
- I plan to build a complete project that combines all the skills I have learned.
- I am interested in learning about Java frameworks and tools used in industry.
- I want to continue practising by solving problems on coding platforms to keep improving.

8. Conclusion

The Object Oriented Programming course has been a very valuable part of my education. I started the course unable to write a single line of Java, and I am finishing it able to design multi-class programs and interactive graphical applications. The progression from week to week was well structured, and each tutorial built on the previous one. Although I faced challenges, especially with inheritance, file handling, and the GUI, I overcame them through practice, research, and asking for help. I now have a solid foundation in both Java programming and object-oriented design, and I am excited to continue learning and applying these skills in future projects.

9. GitHub Repository URL

[OOP LAB Spring-2026](#)